Week 1

Introduction and environment setup

20.10.2026, Tuesdays 11:00–14:00. No graded submission.

This course

5 ECTS, 19.10.2026–11.12.2026. Tuesdays 11:00–14:00.

Machine learning and AI software development on MLOps platforms. In-class sessions, independent exercises, and team work. There is no exam.

Absence from more than 50% of the sessions leads to a failed grade.

One path through the course

ML pipeline, for a specialized model. Evaluate and improve it. Deploy it.

GenAI application, with a general-purpose LLM. Evaluate and improve it.

Use the specialized model as a tool inside that application.

Build the end-to-end system, then evaluate and improve it.

AI-assisted development tools are used throughout. Assessment is about implementing, operating, integrating, and improving.

Exercises on that path

- Week 1. Setup. No graded submission
- Weeks 2–3. Exercise 1, ML pipeline — 15 points. Baseline due Mon 2.11. at 23:59, required to continue
- Weeks 3–4. Exercise 2, GenAI application — 15 points
- Week 5. Exercise 3, model deployment — 20 points
- Week 6. Exercise 4, GenAI and ML integration — 20 points. Team project kickoff
- Weeks 7–8. Exercise 5, end-to-end system — 30 points

Exercises 1–4 are individual. Exercise 5 is the team project.

Today

- Why AI software differs from traditional software
- What the AI and ML lifecycles require
- MLOps, AIOps, and the two platforms in this course
- Environment setup for Exercise 1

Deterministic vs probabilistic

Traditional software follows programmed logic. A small input change has a predictable effect, and a bug can be reproduced.

AI software follows a statistical process. A small input change can move the output a lot. Performance can be measured. It cannot be guaranteed.

Data shifts, and a vendor model can change between runs. GenAI inputs and agent executions vary from run to run.

What "works" means

Traditional acceptance: does it do what the specification says?

AI adds a second question: how often, and how well?

Accuracy and error rate are statistical. The acceptable threshold is a product decision. A recommendation and a diagnosis do not share one.

The lifecycle stays open

Traditional software: release, maintain, deprecate.

AI software: release, monitor, detect degradation, iterate, release again. Cycles are weeks or months.

The same deployment does not repair a degraded model. The process has to notice the change and update the AI part.

Two ways to update

Own model: collect data, retrain or fine-tune, evaluate, redeploy. Slow, and it needs a data pipeline.

Ready model, such as an LLM: change prompts, retrieval, filters, guardrails, or parameters. Faster, and it still needs versions and evaluation.

Both need traceability, evaluation, and controlled rollout.

What you take on operationally

- Store inputs, outputs, and agent traces. Compute metrics. Capture feedback.
- Version the model, prompt, configuration, and the data used to train and evaluate.
- Monitor behaviour. Quality signals and input drift sit beside latency and errors.
- Evaluate on a schedule, including on production inputs.
- Compare the new version with the old one before release.

A bad output should be reconstructable: which version, which context, and whether it is a regression.

MLOps and AIOps

MLOps runs the ML lifecycle: pipelines, experiment and model versions, continuous evaluation, and retraining.

AIOps keeps those foundations and applies them to a whole AI system. Versioning then includes prompts, retrieval, and guardrails. Monitoring adds traces, refusals, and cost.

Exercise 1 is the MLOps path. Later exercises configure and evaluate a GenAI system.

OSS MLOps platform

The ML exercises run on this open-source stack. You use it through an SSH tunnel.

- Kubeflow Pipelines runs the steps
- MLflow stores parameters, metrics, and the run
- KServe is the serving layer, used when the model is deployed

GAIK toolkit

GAIK is a generative AI toolkit for knowledge-centric systems: reusable components, prompts, retrieval, and evaluation.

Exercise 2 starts from a provided GAIK example. Exercise 4 connects that application to the model you deploy.

Laboratory

1. Install JupyterLab Desktop
2. Clone the Exercise 1 repository, branch from `development`, and create `exercise-1-<id>`
3. Open `notebooks/demo-pipeline.ipynb`. Read what each step does, and what compiling the pipeline does
4. Open the SSH tunnel. In Kubeflow and MLflow, browse the demo runs already there

Before next week

No graded submission today.

The branch `exercise-1-<id>` has to exist before the Exercise 1 baseline. Do this setup after the session.

The next session is 27.10.2026. The baseline is due Mon 2.11. at 23:59, and it is required to continue the course.